

ЛР22 (3 пары / 6 часов)

ЛР22 (3 пары / 6 часов)

«CSV: экспорт задач в таблицу и импорт обратно (Excel)»

0) Итог ЛР22 (что должно быть готово)

✓ Добавлены команды:

- 17) Экспорт в CSV
- 18) Импорт из CSV (Replace/Merge)

✓ CSV создаётся в папке `exports`

✓ CSV содержит заголовок:

`Id,Title,Description,Status`

✓ Импорт CSV умеет Replace или Merge

✓ Есть отчёт `docs/LR22_Report.md`

✓ Минимум 3 коммита

1) Теория (важное)

1.1 CSV

CSV — это таблица в текстовом виде, которую открывает Excel.

1.2 Запятые в тексте

Если в тексте есть запятая, её нужно обрабатывать через кавычки `"..."`.

2) Практика. Пара 1 (2 часа) — CsvTaskStorage

Шаг 1. Создать файл CsvTaskStorage.cs

Создать:

- TaskTracker.Storage → Services → CsvTaskStorage.cs

Шаг 2. Вставить код CsvTaskStorage

Вставить полностью:

```
using TaskTracker.Core.Models;

namespace TaskTracker.Storage.Services;

public class CsvTaskStorage
{
    private readonly string _filePath;

    public CsvTaskStorage(string filePath)
    {
        _filePath = filePath;
    }

    public void Save(List<TaskItem> tasks)
    {
        Directory.CreateDirectory(Path.GetDirectoryName(_filePath));

        var lines = new List<string>
        {
            "Id,Title,Description,Status"
        };

        foreach (var t in tasks)
        {
            var id = t.Id.ToString();
            var title = EscapeCsv(t.Title ?? "");
```

```

        var desc = EscapeCsv(t.Description ?? "");
        var status = t.Status.ToString();

        lines.Add($"{id},{title},{desc},{status}");
    }

    File.WriteAllLines(_filePath, lines);
}

public (List<TaskItem> tasks, int errors) Load()
{
    if (!File.Exists(_filePath))
        throw new FileNotFoundException("CSV файл не найден.", _filePath);

    var lines = File.ReadAllLines(_filePath).ToList();
    var result = new List<TaskItem>();
    int errors = 0;

    if (lines.Count == 0)
        return (result, 0);

    int start = 0;
    if (lines[0].StartsWith("Id,", StringComparison.OrdinalIgnoreCaseIgnoreCase))
        start = 1;

    for (int i = start; i < lines.Count; i++)
    {
        var line = lines[i].Trim();
        if (line.Length == 0) continue;

        try
        {
            var parts = ParseCsvLine(line);
            if (parts.Count < 4)

```

```

        {
            errors++;
            continue;
        }

        if (!int.TryParse(parts[0], out var id))
        {
            errors++;
            continue;
        }

        var title = parts[1];
        var desc = parts[2];
        var statusText = parts[3];

        if (!Enum.TryParse<TaskStatus>(statusText, out
t var status))
        {
            if (int.TryParse(statusText, out var sNum
m) && sNum >= 0 && sNum <= 2)
                status = (TaskStatus)sNum;
            else
            {
                errors++;
                continue;
            }
        }

        result.Add(new TaskItem
        {
            Id = id,
            Title = title,
            Description = desc,
            Status = status
        });
    }

```

```

        catch
        {
            errors++;
        }
    }

    return (result, errors);
}

private static string EscapeCsv(string text)
{
    bool mustQuote = text.Contains(',') || text.Contains(
('\"')) || text.Contains('\n') || text.Contains('\r');

    if (text.Contains('\"'))
        text = text.Replace("\"", "\\\"");

    return mustQuote ? $"\"{text}\"" : text;
}

private static List<string> ParseCsvLine(string line)
{
    var result = new List<string>();
    var current = "";
    bool inQuotes = false;

    for (int i = 0; i < line.Length; i++)
    {
        char c = line[i];

        if (inQuotes)
        {
            if (c == '\"')
            {
                if (i + 1 < line.Length && line[i + 1] ==
'\"')

```

```

        {
            current += '"';
            i++;
        }
        else
        {
            inQuotes = false;
        }
    }
    else
    {
        current += c;
    }
}
else
{
    if (c == '"')
    {
        inQuotes = true;
    }
    else if (c == ',')
    {
        result.Add(current);
        current = "";
    }
    else
    {
        current += c;
    }
}
}

result.Add(current);
return result;

```

```
}  
}
```

Шаг 3. Коммит №1

Сообщение:

- `feat: add CsvTaskStorage for export/import`

Commit → Push.

3) Практика. Пара 2 (2 часа) — команды 17/18

Шаг 4. Добавить пункты меню

В `ConsoleUi.PrintMenu()` добавить:

```
Console.WriteLine("17) Экспорт в CSV");  
Console.WriteLine("18) Импорт из CSV (Replace/Merge)");
```

Шаг 5. Команда 17 (Export CSV)

В `Program.cs` добавить:

```
if (input == "17")  
{  
    SafeRunner.Run("EXPORT_CSV", logger, () =>  
    {  
        Directory.CreateDirectory(exportsFolder);  
  
        var csvFile = Path.Combine(  
            exportsFolder,  
            $"tasks_export_{DateTime.Now:yyyy-MM-dd_HH-mm-ss}.csv"  
        );  
  
        var csvStorage = new CsvTaskStorage(csvFile);
```

```

        csvStorage.Save(service.GetAll());

        Console.WriteLine("CSV экспорт выполнен: " + csvFile);
        logger.Info($"EXPORT_CSV file=\"{csvFile}\"");
    });

    continue;
}

```

Шаг 6. Команда 18 (Import CSV)

Добавить обработчик:

```

if (input == "18")
{
    SafeRunner.Run("IMPORT_CSV", logger, () =>
    {
        AccessControl.RequireAdmin(cfg.Role);

        Console.Write("Введите путь к CSV-файлу: ");
        var csvPath = (Console.ReadLine() ?? "").Trim();

        if (string.IsNullOrEmpty(csvPath))
            throw new ArgumentException("Путь пустой.");

        if (!File.Exists(csvPath))
            throw new ArgumentException("Файл не найден: " +
csvPath);

        Console.WriteLine("Режим импорта CSV:");
        Console.WriteLine("1) Replace – заменить текущие зада
чи");
        Console.WriteLine("2) Merge – добавить задачи к текущ
им");

```

```

        Console.WriteLine("Ваш выбор (1/2): ");
        var mode = (Console.ReadLine() ?? "").Trim();
        bool isMerge = mode == "2";

        var csvStorage = new CsvTaskStorage(csvPath);
        var loaded = csvStorage.Load();

        var importedTasks = loaded.tasks;
        var errors = loaded.errors;

        if (isMerge)
        {
            var r = service.MergeImport(importedTasks);
            storage.Save(service.GetAll());

            Console.WriteLine($"CSV Merge: добавлено {r.added}, пропущено {r.skipped}, ошибок строк {errors}");
            logger.Info($"IMPORT_CSV_MERGE added={r.added} skipped={r.skipped} errors={errors}");
        }
        else
        {
            service.ReplaceAll(importedTasks);
            storage.Save(service.GetAll());

            Console.WriteLine($"CSV Replace: загружено {importedTasks.Count}, ошибок строк {errors}");
            logger.Info($"IMPORT_CSV_REPLACE count={importedTasks.Count} errors={errors}");
        }
    });

    continue;
}

```

Шаг 7. Коммит №2

Сообщение:

- `feat: add CSV export/import commands`

Commit → Push.

4) Практика. Пара 3 (2 часа) — тест через Excel и отчёт

Шаг 8. Проверка экспорта CSV

1. Добавить 2–3 задачи
 2. Нажать 17
 3. Открыть файл `.csv` в Excel
 4. Убедиться, что таблица корректная
-

Шаг 9. Проверка импорта CSV

1. Изменить Title в Excel
 2. Сохранить CSV
 3. Нажать 18
 4. Выбрать Replace или Merge
 5. Проверить список задач
-

Шаг 10. Отчёт `docs/LR22_Report.md`

Создать файл:

```
# Отчёт ЛР22 – CSV
```

```
## Что сделано
```

- Добавлен CsvTaskStorage для чтения/записи CSV
- Добавлены команды:

- 17) Экспорт в CSV
- 18) Импорт из CSV (Replace/Merge)
- Импорт считает ошибки строк и не падает

Как проверить

- 1) Создать 2–3 задачи
- 2) Экспортировать в CSV (17)
- 3) Открыть CSV в Excel и изменить одну задачу
- 4) Импортировать CSV (18) в режиме Replace или Merge
- 5) Проверить список задач

Шаг 11. Коммит №3

Сообщение:

- `docs: add LR22 report`

Commit → Push.

5) Что сдавать по ЛР22

- ✓ пункты 17/18 в меню
- ✓ экспорт CSV в `exports`
- ✓ импорт Replace/Merge
- ✓ логирование
- ✓ `docs/LR22_Report.md`
- ✓ минимум 3 коммита

6) Оценивание

“3” — экспорт CSV работает и открывается в Excel

“4” — + импорт Replace

“5” — + импорт Merge + логирование и ошибки строк